

Requirement Analysis Document

To-Do Application

Document Version: 1.0

Date: April 3, 2026

Status: Final

Author: Requirements Analysis Team

Table of Contents

1. [Introduction](#)
 2. [SRS Summary](#)
 3. [Functional Requirements](#)
 4. [Non-Functional Requirements](#)
 5. [User Roles & Stakeholders](#)
 6. [Data Entities & Relationships](#)
 7. [Constraints & Assumptions](#)
 8. [Technology Stack Recommendations](#)
-

1. Introduction

1.1 Project Overview

The To-Do Application is a lightweight task management solution designed for general users across mobile and web platforms. The application enables users to create, view, complete, and delete personal tasks through a deliberately simplified interface. This analysis document translates the high-level Software Requirements Specification into detailed, actionable requirements suitable for development, testing, and project management.

1.2 Purpose of This Document

This Requirement Analysis Document (RAD) serves to: - Decompose high-level requirements into specific, testable functional and non-functional requirements - Establish a unified understanding among stakeholders, developers, and testers - Define data models and system boundaries - Identify constraints and assumptions that impact technical decisions - Provide a foundation for architectural design and implementation planning

1.3 Scope

In-Scope: - Core task lifecycle management (create, read, update, delete) - Cross-platform accessibility (web browsers and mobile devices) - Persistent local data storage - Basic user interface for task interaction

Out-of-Scope: - User authentication and multi-user support - Task categories, tags, or advanced organization - Collaboration or sharing features - Notification systems or reminders - Cloud synchronization - Advanced analytics or reporting - Integration with external services

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
Task	A single unit of work consisting of a title, optional description, and completion status

CRUD	Create, Read, Update, Delete - fundamental data operations
PWA	Progressive Web Application - web app with native-like capabilities
MoSCoW	Prioritization method: Must-have, Should-have, Could-have, Won't-have
NFR	Non-Functional Requirement

2. SRS Summary

2.1 Executive Summary

The To-Do Application addresses the fundamental need for digital task management through a minimal viable product (MVP) approach. With a complexity estimate of **Low**, the project prioritizes speed-to-market and user simplicity over feature richness. The system comprises four core functional requirements and two critical non-functional requirements, serving a single user role across multiple platforms.

2.2 Key Objectives

1. **Simplicity:** Deliver an interface that requires zero learning curve
2. **Performance:** Achieve sub-second response times for all user interactions
3. **Reliability:** Maintain 99.9% data integrity with zero unrecoverable data loss
4. **Accessibility:** Support modern web browsers and mobile OS versions released within the last 3 years

2.3 Success Criteria

Criterion	Measurement
User Onboarding	100% of test users can add first task within 30 seconds
Task Operations	All CRUD operations complete in <500ms on target devices
Data Persistence	Zero data loss during normal app lifecycle
UI Simplicity	Interface contains ≤ 5 primary interactive elements

3. Functional Requirements

FR1: Create Task

Requirement ID: FR-001

Priority: Must-have

Description: The system shall allow users to create new tasks with a title and optional description.

Detailed Specification: - Users must be able to access the "Add Task" function from the main view - The task creation interface shall contain: - **Title field:** Required, text input, maximum 200 characters - **Description field:** Optional, multi-line text input, maximum 1000 characters - The system shall validate that title is not empty or whitespace-only - Upon creation, tasks shall receive a unique identifier and default status of "Pending" - The new

task shall immediately appear in the user's task list

Acceptance Criteria:

Given a user on the main task view
When the user enters "Buy groceries" in the title field
And clicks the "Add Task" button
Then the task "Buy groceries" appears at the top of the task list
And the title field is cleared for next input

User Story:

As a user, I want to quickly add tasks so that I can capture to-do items as they come to mind.

FR2: View Tasks

Requirement ID: FR-002

Priority: Must-have

Description: The system shall display all tasks in a scrollable list with visual distinction between pending and completed states.

Detailed Specification: - The main view shall display tasks in reverse chronological order (newest first) - Each task item shall show: - Title (truncated after 50 characters if necessary) - Completion status indicator (checkbox or similar) - Creation date in relative format (e.g., "Today", "Yesterday") - The list shall support vertical scrolling for content overflow - Completed tasks shall appear with strikethrough text and reduced opacity - The system shall display an empty state message when no tasks exist

Acceptance Criteria:

Given a user with 3 existing tasks (2 pending, 1 completed)
When the user opens the application
Then all 3 tasks are visible in the list

And completed task is visually distinct from pending tasks
And tasks are ordered by creation date descending

User Story:

As a user, I want to see all my tasks in one place so that I can understand what needs to be done.

FR3: Update Task Status

Requirement ID: FR-003

Priority: Must-have

Description: The system shall allow users to toggle a task's status between "Pending" and "Completed".

Detailed Specification: - Each task item shall include a clickable completion toggle control - Clicking the toggle shall: - Change the task status to "Completed" if currently "Pending" - Change the task status to "Pending" if currently "Completed" - Record the completion timestamp when marked complete - Visual feedback shall be provided within 100ms of user interaction - Status changes shall persist immediately without requiring explicit save action

Acceptance Criteria:

Given a user viewing a pending task "Finish report"
When the user clicks the completion checkbox
Then the task displays as completed with strikethrough styling
And the task persists as completed after app restart

User Story:

As a user, I want to mark tasks complete so that I can track my progress.

FR4: Delete Task

Requirement ID: FR-004

Priority: Must-have

Description: The system shall allow users to permanently delete tasks with a confirmation step.

Detailed Specification: - Each task item shall include a delete affordance (trash icon or swipe gesture) - Upon delete initiation, the system shall display a confirmation dialog - The confirmation dialog shall contain: - Clear warning message: "Delete this task permanently?" - "Cancel" button to abort the operation - "Delete" button to confirm removal - Deleted tasks shall be irrecoverably removed from storage - The task list shall refresh immediately after deletion

Acceptance Criteria:

Given a user viewing a task "Old meeting notes"
When the user initiates delete and confirms in the dialog
Then the task is removed from the list
And the task does not reappear after app restart

User Story:

As a user, I want to delete tasks so that I can remove irrelevant or erroneous items.

4. Non-Functional Requirements

NFR1: Performance

Requirement ID: NFR-001

Category: Performance Efficiency

Priority: Must-have

Sub-Requirement	Measurement Criteria
Initial Load Time	Application must render first interactive content in < 2 seconds on 3G connection
Task Operations	All CRUD operations must complete in < 500ms on target device (iPhone 8 equivalent)
List Rendering	Task lists with up to 100 items must scroll at 60fps without frame drops
Memory Footprint	Application must use < 50MB RAM in idle state

Rationale: User engagement drops 53% if load time exceeds 3 seconds. The simplicity constraint demands instant responsiveness.

NFR2: Data Security & Integrity

Requirement ID: NFR-002
Category: Security & Reliability
Priority: Must-have

Sub-Requirement	Measurement Criteria
-----------------	----------------------

Data Persistence	100% of user data must survive application crashes and device restarts
Storage Encryption	All task data must be encrypted at rest using platform-native encryption (e.g., iOS Keychain, Web Crypto API)
Data Validation	System must reject malformed data entries and prevent XSS/SQL injection through input sanitization
Backup Resilience	Local storage must implement atomic write operations to prevent corruption during power loss

Rationale: While the app is single-user, data safety is non-negotiable for user trust.

NFR3: Usability

Requirement ID: NFR-003

Category: Usability

Priority: Should-have

- **Learnability:** New users must achieve task creation within 30 seconds without documentation
- **Accessibility:** UI must meet WCAG 2.1 AA standards for contrast and keyboard navigation
- **Error Rate:** < 5% of user actions result in unintended outcomes
- **Touch Target Size:** All interactive elements must be $\geq 44 \times 44$ pt on mobile devices

NFR4: Compatibility

Requirement ID: NFR-004

Category: Compatibility

Priority: Should-have

Platform	Minimum Version	Supported Browsers
iOS	14.0+	Safari, Chrome
Android	9.0 (API 28)+	Chrome, Samsung Internet
Web	N/A	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

5. User Roles & Stakeholders

5.1 Primary Users

Role	Description	Key Needs
------	-------------	-----------

General User	Individual seeking basic task management without complexity	Speed, simplicity, reliability
--------------	---	--------------------------------

User Personas

Persona 1: Quick Capture Claire - *Demographics:* 28-year-old marketing professional - *Goals:* Rapidly add tasks during meetings and commutes - *Frustrations:* Complex apps with too many features - *Primary Use Case:* Add 5-10 tasks daily, mark complete by end of day

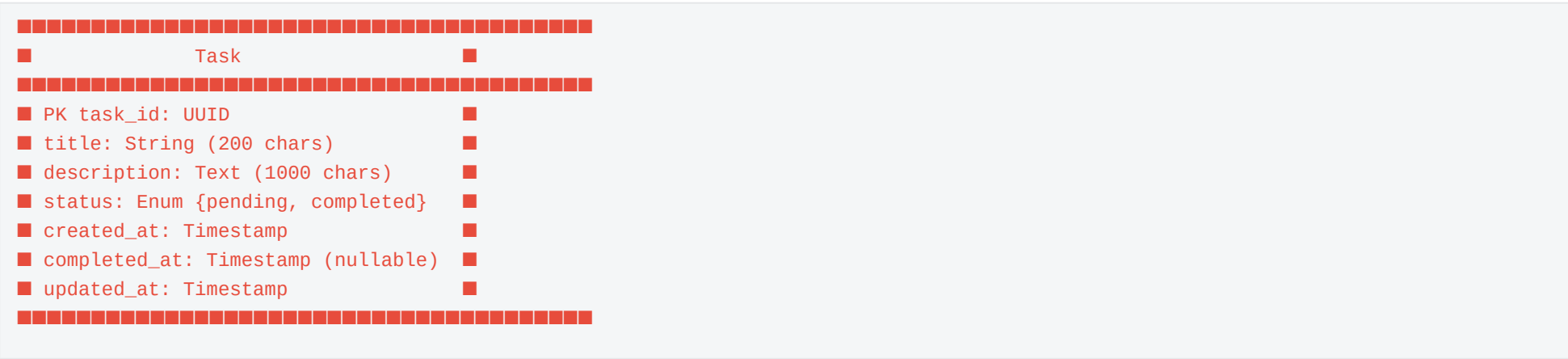
Persona 2: Minimalist Mike - *Demographics:* 45-year-old freelancer - *Goals:* Maintain a single, uncluttered task list - *Frustrations:* Cluttered interfaces and learning curves - *Primary Use Case:* Weekly planning with simple CRUD operations

5.2 Secondary Stakeholders

Stakeholder	Interest	Impact
Development Team	Clean architecture, maintainable code	High
Product Owner	On-time delivery, scope adherence	High
QA Tester	Testable requirements, clear acceptance criteria	Medium
Support Team	Minimal support overhead due to simplicity	Low

6. Data Entities & Relationships

6.1 Entity Relationship Diagram



6.2 Data Dictionary

Attribute	Data Type	Constraints	Default Value	Description
task_id	UUID	Primary Key, Unique, Not Null	Auto-generated	Unique task identifier
title	String(200)	Not Null, Trim Whitespace	N/A	Brief task description

<code>description</code>	Text(1000)	Nullable	Null	Detailed task notes
<code>status</code>	Enumeration	Not Null, Values: <code>pending, completed</code>	<code>pending</code>	Current completion state
<code>created_at</code>	Timestamp	Not Null, Auto-set	CURRENT_TIMESTAMP	Creation timestamp
<code>completed_at</code>	Timestamp	Nullable	Null	Completion timestamp
<code>updated_at</code>	Timestamp	Not Null, Auto-update	CURRENT_TIMESTAMP	Last modification timestamp

6.3 Data Volume Estimates

- **Average tasks per user:** 25 active, 75 completed (100 total)
- **Storage per task:** ~2KB (including metadata)
- **Total storage per user:** ~200KB
- **Growth rate:** 10 tasks/week average

7. Constraints & Assumptions

7.1 Technical Constraints

Constraint ID	Description	Impact
TC-001	Simple UI: Interface must contain no more than 5 primary interactive elements	Limits feature creep; enforces minimalism
TC-002	No Advanced Features: Excludes categories, tags, reminders, search, or sorting options	Reduces development scope but may limit user flexibility
TC-003	Platform Agnostic: Must support both web and mobile from single codebase (if possible)	Influences technology stack selection
TC-004	Offline-First: Application must function without internet connectivity	Requires local storage architecture

7.2 Business Constraints

- **Budget:** Low-cost development and maintenance (suitable for MVP)
- **Timeline:** Development sprint ≤ 4 weeks
- **Team Size:** 1-2 developers maximum
- **Licensing:** Only open-source or free-tier technologies permitted

7.3 Assumptions

Assumption ID	Description	Risk if Invalid
AS-001	Users have basic digital literacy and understand checkbox metaphors	High - requires onboarding if false
AS-002	Target devices have >100MB available storage	Low - modern devices exceed this
AS-003	No regulatory compliance required (GDPR, HIPAA) for personal task data	Medium - may need data export feature
AS-004	Application will be used by single user per device; no account system needed	High - changes architecture if multi-user needed
AS-005	Average task lifecycle is <30 days (short-term tasks)	Low - impacts data volume only

8. Technology Stack Recommendations

Based on the low complexity, cross-platform requirement, and simplicity constraints, three viable architecture options are presented:

8.1 Option A: Progressive Web Application (PWA) - Recommended

Best for: Broadest reach, single codebase, lowest cost

Layer	Technology	Justification
Frontend	React 18 + Vite	Fast build times, excellent PWA support, small bundle size
UI Framework	Tailwind CSS + Headless UI	Rapid styling, accessibility-compliant components
State Management	Zustand + IndexedDB	Lightweight, persistent local storage, no backend needed
PWA Framework	Vite PWA Plugin	Automated service worker generation, offline support
Testing	Vitest + Playwright	Fast unit tests, cross-browser E2E testing
Deployment	Vercel / Netlify	Free tier, automatic deployments, global CDN

Pros: Instant updates, no app store approval, works offline

Cons: Limited iOS native features, requires browser

8.2 Option B: Cross-Platform Mobile (React Native)

Best for: Native mobile experience, app store distribution

Layer	Technology	Justification
Framework	React Native 0.73	Single codebase for iOS/Android, mature ecosystem
Storage	AsyncStorage + SQLite	Persistent local storage, simple API
UI	React Native Paper	Material Design, accessible components
Navigation	React Navigation	Standard routing solution
Build Tool	Expo EAS	Simplified builds and deployment
Testing	Jest + Detox	Unit and E2E testing

Pros: Native performance, app store presence, push notification ready (future)

Cons: Requires separate web version if needed, larger binary size

8.3 Option C: Native Mobile (Swift + Kotlin)

Best for: Maximum performance, platform-specific UX

Platform	Technology	Justification
iOS	SwiftUI + SwiftData	Modern declarative UI, native persistence
Android	Jetpack Compose + Room	Modern toolkit, type-safe persistence
Shared Logic	Kotlin Multiplatform	Share business logic, keep native UI

- Pros:** Best performance, pure native experience
- Cons:** Highest development effort, 2x codebase maintenance

8.4 Database Selection Matrix

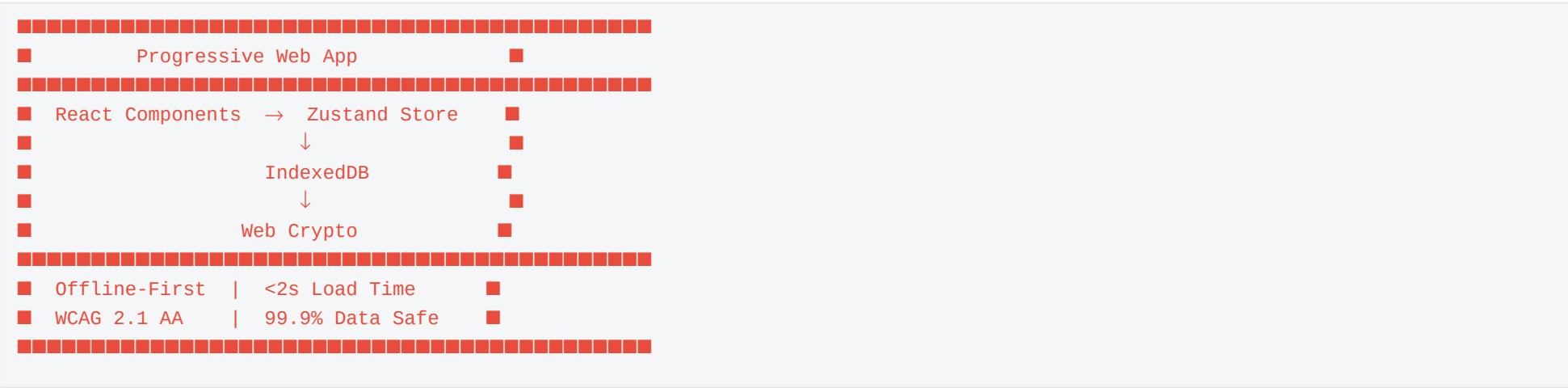
Criteria	IndexedDB (PWA)	SQLite (Mobile)	SwiftData/Room (Native)
Complexity	Low	Medium	Low
Encryption	Web Crypto API	SQLCipher	Native encryption
Query Speed	Fast (<50ms)	Very Fast (<20ms)	Very Fast (<20ms)

Data Volume	Up to 500MB	Unlimited	Unlimited
Recommendation	■ PWA	■ Cross-Platform	■ Native

8.5 Final Recommendation

For MVP with maximum reach: Adopt **Option A (PWA)** using React + Vite + IndexedDB. This satisfies all constraints, delivers sub-second load times, and enables future mobile wrapping via Capacitor if native distribution becomes necessary.

Architecture Diagram:



Appendix A: Requirement Traceability Matrix

Requirement	Source	Priority	Verification Method
FR-001	SRS §3	Must	Unit Test + E2E Test
FR-002	SRS §3	Must	Visual Test + E2E Test
FR-003	SRS §3	Must	Unit Test + E2E Test
FR-004	SRS §3	Must	E2E Test
NFR-001	SRS §4	Must	Performance Monitoring
NFR-002	SRS §4	Must	Security Audit
NFR-003	Derived	Should	Usability Testing
NFR-004	Derived	Should	Compatibility Testing

Document Control:

Approved by: Product Owner

Reviewed by: Technical Lead, QA Lead

Next Review Date: Post-MVP Launch